COS40007 Artificial Intelligence Engineering

Applied Project — Group Task 1

System Blueprint & Project Planning

Field	Details
Unit Code	COS40007 — Artificial Intelligence Engineering
Task	Group Task 1: System Blueprint & Project Planning
Semester	2026, Semester 1
Institution	Swinburne University of Technology
Group Number	CL06_G03
Submission Date	15 April 2026
GitHub Repository	https://classroom.github.com/a/OF9PeW7N
Trello Board	https://trello.com/b/jOfvz8HH/cl06g03-ai-for-engineering
Shared drive	https://liveswinburneeduau-my.sharepoint.com/:w:/r/personal/105321162_student_swin_edu_au/_layouts/15/Doc.aspx?sourcedoc=%7B0B1CCAFA-FA1D-479D-A64B-5AB82B017CD1%7D&file=COS40007_Group_Task1_Final%201.docx&action=default&mobileredirect=true

Team Members

Name	Student ID	Role
Sahil	105295665	Project Manager / Scrum Master
Seniru	104786647	ML Engineer
Parav	105321434	Data Engineer
Nainika	105321162	MLOps / DevOps Lead

Section 1: Group Agreement & Project Plan

1.1 Project Overview

This project develops an AI-Powered Obesity Level Classification System for health and wellness applications. The system analyses individual eating habits and physical condition data — including age, weight, height, dietary patterns, and physical activity levels — to classify a person's obesity level into one of seven categories: Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II, Obesity Type I, Obesity Type II, and Obesity Type III. The outcome enables healthcare providers to identify at-risk individuals, support early intervention, and build personalised nutrition and fitness recommendation systems.

1.2 Team Roles & Responsibilities

Member	Role	Responsibilities
Sahil	Project Manager / Scrum Master	Sprint planning, Trello management, team meeting coordination, report compilation and submission, overall architecture oversight
Seniru	ML Engineer	ML model selection and training (Random Forest, Decision Tree), hyperparameter tuning, model evaluation, Flask REST API development, MLflow experiment tracking
Parav	Data Engineer	Dataset sourcing from UCI ML Repository, data cleaning, EDA on eating habits features, preprocessing pipeline, feature encoding, class balancing
Nainika	MLOps / DevOps Lead	GitHub repository setup, branching strategy, CI/CD pipeline design, MLflow model registry, Evidently AI drift detection, deployment pipeline

1.3 Communication & Collaboration Strategy

- Primary communication: WhatsApp group for daily updates and quick decisions
- Meetings: Twice per week — one session during studio, one online via Zoom
- Document sharing: Microsoft Teams for file sharing and longer discussions
- Task tracking: Trello board updated after every meeting to reflect current progress
- Version control: All code and documents pushed to GitHub Classroom repository

1.4 Conflict Resolution

All disagreements are raised openly during team meetings and resolved by majority vote. If a disagreement cannot be resolved within 48 hours, the Project Manager (Sahil) makes the final decision. Persistent unresolved conflicts are escalated to the unit tutor.

1.5 Sprint Plan

Sprint 1 — DevOps Setup & Data Preparation (Weeks 5–7)

Element	Details
Sprint Goal	Establish project infrastructure, collect and prepare obesity dataset, and set up experiment tracking
Key Deliverables	GitHub repository with branch structure, Trello board configured, UCI obesity dataset downloaded, EDA notebook completed, preprocessing pipeline built, MLflow experiment tracking initialised
Team Allocation	Parav: data collection and EDA; Nainika: GitHub and MLflow setup; Seniru: baseline classification model training; Sahil: sprint coordination and Trello
Success Criteria	Data pipeline operational; GitHub branches established; baseline model achieves >80% AUC-ROC; all Trello tasks assigned
Key Risks	Some categorical features require encoding; class distribution across 7 obesity levels needs verification

Table 1.5.1 — This table demonstrates the team's view on Sprint 01.

Sprint 2 — Model Training & Deployment (Weeks 8–9)

Element	Details
Sprint Goal	Train an optimised ML model, register it in MLflow, and deploy it as a REST API
Key Deliverables	Trained Random Forest and Decision Tree models, MLflow model registry entry with version tags, Flask REST API with /predict endpoint, GitHub Actions CI/CD workflow, progress report
Team Allocation	Seniru: model training and API development; Nainika: CI/CD pipeline and model registry; Parav: feature engineering; Sahil: progress report and coordination
Success Criteria	Model achieves target precision and recall on test set; API deployed and tested; CI/CD pipeline executes automatically on push to develop branch
Key Risks	Minor class imbalance across 7 obesity levels; model may favour majority classes

Table 1.5.2 — This table demonstrates the team's view on Sprint 02.

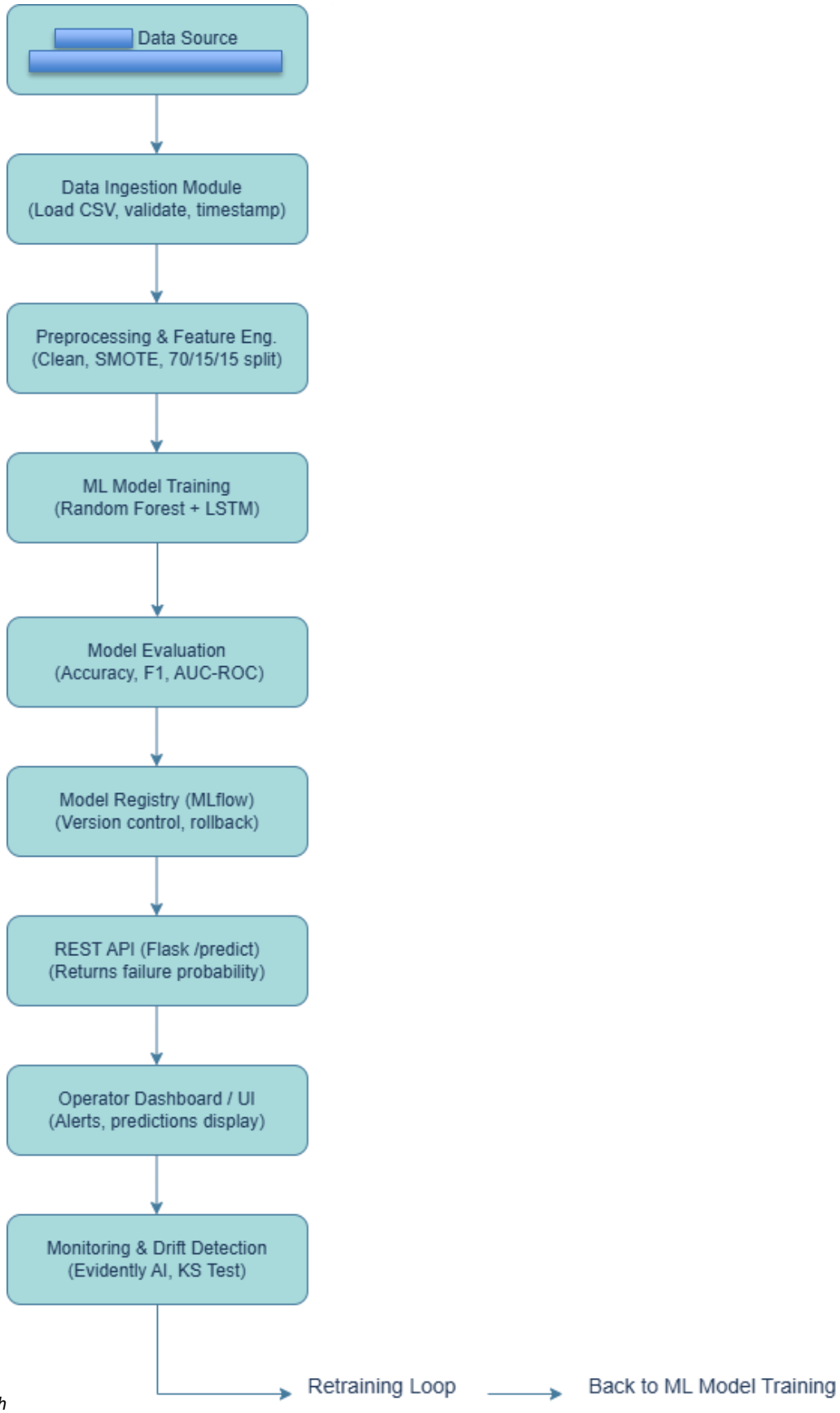
Sprint 3 — Monitoring, Drift Detection & Final Report (Weeks 10–11)

Element	Details
Sprint Goal	Implement post-deployment monitoring, drift detection, automated retraining pipeline, and prepare all final deliverables
Key Deliverables	Evidently AI drift detection reports, automated retraining trigger, user-facing monitoring dashboard, demo video, final project report
Team Allocation	Nainika: monitoring and drift detection; Seniru: retraining pipeline; Parav: data validation; Sahil: final report writing and demo coordination
Success Criteria	Drift alerts trigger correctly; retraining pipeline runs when threshold is exceeded; monitoring dashboard is live and displaying predictions
Key Risks	Delayed ground truth labels for concept drift evaluation; alert fatigue from excessive false alarm triggers

Table 1.5.3 — This table demonstrates the team's view on Sprint 03.

Section 2: System Blueprint Diagram

2.1 Architecture Diagram



sh *Image 2.1.1 — This*

image demonstrates the Architecture Diagram of the Obesity Level Classification AI system.

2.2 System Data Flow

The following describes the end-to-end data flow of the Obesity Level Classification AI system:

```
┌──────────────────────────────────────┐
│          Input Data Sources          │
│  (Age, Gender, Height, Weight,       │
│   Eating Habits, Physical Activity,  │
│   Family History)                    │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│        Data Ingestion Module         │
│   (Load CSV, validate schema,        │
│    validate feature types, encode    │
│    categorical variables)            │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│   Preprocessing & Feature Engineering │
│   (Label encoding, standard scaling, │
│    BMI engineering, class weights,   │
│    stratified 70/15/15 data split)   │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│        ML Model Training             │
│   (Random Forest + Decision Tree;    │
│    all runs logged in MLflow)        │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│      Model Evaluation & Validation   │
│  (Precision, Recall, F1, AUC-ROC,    │
│   Confusion Matrix)                  │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│      Model Registry — MLflow         │
│  (Version tags, metadata, rollback   │
│   support, performance history)      │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│   REST API Serving Layer — Flask     │
│  (POST /predict → returns            │
│   obesity_level + confidence_score)  │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│      Healthcare Dashboard / UI       │
│  (Real-time predictions, confidence  │
│   scores, health risk alerts)        │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│   Monitoring & Drift Detection       │
│   (Evidently AI weekly reports,      │
│    KS-test on Age, Weight, BMI,      │
```

```
│    concept drift via accuracy monitoring)│
└──────────────────────────────────────────┘
                    │
        (if drift or performance drop)
                    │
                    └──────→ Retraining Trigger
                              → back to ML Model Training
```

2.3 Component Descriptions

Input Data Sources

The dataset used is the Estimation of Obesity Levels Based on Eating Habits and Physical Condition dataset (Palechor & De la Hoz Manotas, 2019), sourced from the UCI Machine Learning Repository (DOI: 10.24432/C5H31Z). It contains 2,111 records and 16 features including gender, age, height, weight, family history of overweight, frequency of high-calorie food consumption, vegetable intake, number of meals, physical activity frequency, water intake, and transport mode. The target variable NObesity has 7 class labels ranging from Insufficient Weight to Obesity Type III.

Data Ingestion Module

Python scripts load the raw CSV file from the UCI repository, validate schema and feature types, check for class distribution across all 7 obesity levels, and pass clean data to the preprocessing stage. Managed by Parav in Sprint 1.

Preprocessing & Feature Engineering

Handles label encoding for categorical variables (Gender, CAEC, CALC, MTRANS), applies standard scaling to numerical features (Age, Height, Weight, BMI), and engineers a derived BMI feature (Weight ÷ Height^2). Class weights are applied to address minor imbalance across the 7 obesity level classes. Data is split into 70% training, 15% validation, and 15% test using stratified sampling to preserve class proportions.

ML Model Training

Two models are trained and compared: Random Forest (interpretable, handles mixed categorical and numerical health data) and Decision Tree (clinician-friendly, produces visual decision rules). All experiment runs — including hyperparameters, evaluation metrics, and model artefacts — are tracked in MLflow by Seniru.

Model Evaluation & Validation

Model performance is assessed using precision, recall, F1-score, and multi-class AUC-ROC. Confusion matrices are used to identify misclassifications between adjacent obesity levels (e.g., Overweight Level I vs Overweight Level II). Only models meeting the minimum F1-score threshold of 85% are promoted to the registry.

Model Registry — MLflow

Stores all approved model versions with version tags (e.g., v1.0-sprint2), training metadata, evaluation scores, and model artefact paths. Enables instant comparison between model versions and rollback to any previously approved model.

REST API Serving Layer — Flask

The best-performing model is wrapped in a Flask application exposing a /predict endpoint. The API accepts a person's eating habits and physical condition data as a POST request and returns a JSON response containing obesity_level (one of 7 class labels) and confidence_score (0–1 probability). Latency target is under 5 seconds at the 99th percentile.

Healthcare Dashboard / UI

A web-based dashboard displaying real-time obesity level predictions, confidence scores, and health alerts for each user profile. Healthcare providers can view prediction history and act on high-risk classifications. Built in Sprint 3.

Monitoring & Drift Detection — Evidently AI

Post-deployment monitoring compares live input feature distributions to training data weekly using Evidently AI. The Kolmogorov-Smirnov test is applied to numerical features (Age, Weight, BMI) to detect data drift. Concept drift is detected by monitoring model accuracy on newly collected user health data. If accuracy drops more than 5% over a 2-week rolling window, the retraining pipeline is triggered automatically.

Section 3: MLOps & DevOps Integration

3.1 Data Split & Training Strategy

The dataset is split using stratified sampling to reflect real-world deployment conditions and ensure all 7 obesity classes are proportionally represented in each split:

Split	Proportion	Purpose
Training Set	70%	Fit model parameters
Validation Set	15%	Hyperparameter tuning and model selection
Test Set	15%	Final held-out evaluation only

Stratified sampling is used to ensure all 7 obesity classes are proportionally represented in each split. As new user health records become available during deployment, they are versioned using DVC and incorporated into the next retraining cycle, ensuring the model adapts to changing population health behaviour over time.

3.2 Retraining Strategy

Trigger	Condition	Response
Performance-based	Classification F1-score drops below 85% on validation set	Immediate retraining pipeline triggered
Time-based	Every 2 weeks during Sprints 2–3	Scheduled retraining runs automatically
Drift-based	KS-test p-value < 0.05 on key features	Alert raised; retraining initiated within 24 hours

3.3 Monitoring & Drift Detection

Operational Metrics (tracked per inference request):

- Prediction confidence score distribution
- Output class frequency across 7 obesity levels
- API response latency (target: under 5 seconds)
- System error rate per session

Data Drift Detection:

Data drift occurs when the statistical distribution of incoming input features diverges from the training data distribution (e.g., due to changing demographics or shifts in user population behaviour). Detection approach:

- KS-test applied to Age, Weight, BMI, and physical activity features

- Evidently AI generates automated weekly drift reports comparing live feature distributions to training baseline

Concept Drift Detection:

Concept drift occurs when the relationship between eating habits and obesity outcomes changes (e.g., due to population lifestyle shifts). Detection approach:

- Monitor model accuracy on newly collected user health data batches
- Alert threshold: accuracy drop exceeding 5% over a 2-week rolling window triggers the retraining pipeline

3.4 DevOps Practices

Version Control — GitHub

All code is managed in the GitHub Classroom repository using the following branching strategy:

Branch	Purpose
main	Stable, production-ready code only — no direct pushes
develop	Active development — all feature branches merge here
feature/[name]	Individual feature branches (e.g., feature/preprocessing-pipeline)

All changes are merged via pull requests with peer review. Managed by Nainika.

CI/CD Pipeline — GitHub Actions

A GitHub Actions workflow runs automatically on every push to the develop branch:

1. Execute unit tests on preprocessing and model training scripts
2. Validate dataset integrity (schema, null values, class distribution)
3. Trigger model retraining if performance drift threshold is exceeded
4. Deploy updated model to the REST API serving layer if all tests pass

(Full implementation is planned for Sprint 2. This section represents the conceptual design.)

Code & Model Versioning

Asset	Versioning Method
Code	Git tags per sprint release (e.g., v1.0-sprint1, v2.0-sprint2)
Data	DVC tracks all dataset versions with checksums, linked to GitHub
Models	MLflow logs every experiment run with parameters,

	metrics, and artefact paths

Section 4: Project Management Setup

4.1 Project Management Tool

Tool: Trello

Board Name: CL06_G03 — AI For Engineering

Board Link: https://trello.com/b/jOfvz8HH/cl06g03-ai-for-engineering

4.2 Trello Board Screenshot

Image 4.2 — This image shows the Trello Board representing the team's task distribution and sprint progress.

4.3 User Stories

US-01 — Data Preprocessing & Encoding

As a data engineer, I want to clean and encode the UCI obesity dataset so that the machine learning model can correctly process all 16 features including categorical variables.

Scenario	Given	When	Then
Encoding completes	I have the raw CSV from UCI	I run the preprocessing script	All categorical features are encoded and numerical features are scaled
Class verified	Dataset is loaded	I check class distribution	All 7 obesity levels are present with counts recorded
Split complete	Preprocessing is done	I run the train/test split	70/15/15 split with stratification is saved to

			the /data folder

Definition of Done: Preprocessed dataset committed to GitHub; EDA notebook showing class distribution completed.

US-02 — Baseline Model Training

As an ML engineer, I want to train a baseline predictive model so that I can establish initial performance benchmarks and identify areas for improvement.

Scenario	Given	When	Then
Training completes	I have a preprocessed and labelled dataset	I run the training script	The Random Forest model achieves >80% AUC-ROC on the validation set
Experiment logged	The model is trained	I check the MLflow dashboard	All hyperparameters, evaluation metrics, and model artefacts are logged for that run

Definition of Done: Baseline model saved to the MLflow model registry; training script committed to GitHub.

US-03 — CI/CD Pipeline Setup

As a DevOps lead, I want a GitHub Actions CI/CD pipeline so that all code changes are automatically tested before merging into the main branch.

Scenario	Given	When	Then
Code pushed	A developer pushes code to the develop branch	The push triggers the GitHub Actions workflow	Unit tests run automatically and results are reported in the pull request
All tests pass	All tests pass in the pipeline	The pull request is reviewed	Code is automatically eligible for merge without manual testing
Test failure	A test fails in the pipeline	The pipeline runs	The merge is blocked and the developer receives an automated notification

Definition of Done: GitHub Actions workflow file committed to the repository; minimum 3 automated tests passing.

US-04 — Project Task Tracking

As a project manager, I want a Trello board with all tasks assigned to team members so that sprint progress is visible to everyone and accountability is maintained.

Scenario	Given	When	Then
Sprint begins	The project enters Sprint 1	I open the Trello board	All tasks are listed with assigned owners, sprint labels, and current status

Definition of Done: Trello board screenshot included in the Task 1 report; all members have at least one card assigned.

US-05 — Real-Time Prediction API

As an ML engineer, I want to deploy the trained obesity classification model as a low-latency REST API so that the healthcare platform can request obesity level predictions in real time.

Scenario	Given	When	Then
API request received	The API receives a person's eating habits and physical condition data	A POST request is sent to /predict	The API returns obesity_level (one of 7 class labels) and confidence_score (0–1 probability)
Performance under load	The API is under concurrent load	100 simultaneous requests are sent	99th percentile latency remains under 5 seconds

Definition of Done: API deployed to staging environment; load tested with 100 concurrent users; Swagger documentation available.

US-06 — Drift Detection & Automated Retraining

As an MLOps lead, I want drift detection configured and connected to the retraining pipeline so that model performance degradation is identified and corrected automatically without manual intervention.

Scenario	Given	When	Then
Data drift detected	The model is live and receiving user health data	Input feature distributions shift (KS p-value < 0.05)	An alert is raised and the retraining pipeline is triggered within 24 hours
Concept drift detected	Model accuracy is monitored on labelled batches	Accuracy drops more than 5% over 2 weeks	Automated retraining is initiated using the latest available labelled data

Definition of Done: Evidently AI drift reports generated weekly; retraining trigger threshold documented, configured, and tested.

4.4 Product Backlog

#	Task	Owner	Sprint	Status
1	Set up GitHub repo and branch structure	Nainika	Sprint 1	Done
2	Create and configure Trello board	Sahil	Sprint 1	Done
3	Download UCI Obesity dataset (ID: 544)	Parav	Sprint 1	In Progress
4	Perform Exploratory Data Analysis (EDA)	Parav	Sprint 1	To Do
5	Encode categorical features and engineer BMI feature	Parav	Sprint 1	To Do
6	Preprocess data and apply class weights	Parav	Sprint 1	To Do
7	Set up MLflow experiment tracking	Nainika	Sprint 1	To Do
8	Train baseline Random Forest model	Seniru	Sprint 1	To Do
9	Train Decision Tree model and compare performance	Seniru	Sprint 2	To Do
10	Hyperparameter tuning and model optimisation	Seniru	Sprint 2	To Do
11	Configure GitHub Actions CI/CD pipeline	Nainika	Sprint 2	To Do
12	Build Flask REST API /predict endpoint	Seniru	Sprint 2	To Do
13	Deploy API to staging environment	Nainika	Sprint 2	To Do
14	Implement Evidently AI drift detection	Nainika	Sprint 3	To Do
15	Build healthcare monitoring dashboard	All	Sprint 3	To Do
16	Implement automated	Nainika	Sprint 3	To Do

	retraining pipeline			
17	Record project demo video	All	Sprint 3	To Do
18	Write and submit final project report	All	Sprint 3	To Do

4.5 Project Documentation

GitHub README.md contents:

- Project title and description
- Team member names and roles
- System architecture overview
- Setup and installation instructions
- Links to Trello board and report

Key Design Decisions:

Decision	Choice	Rationale
Primary model	Random Forest	Interpretable, handles mixed categorical and numerical health data, low inference latency
Comparison model	Decision Tree	Clinician-friendly; produces visual decision rules for health data interpretation
Dataset	UCI Obesity Dataset — Palechor & De la Hoz Manotas (2019)	2,111 records, 7-class target, no missing values, CC BY 4.0 licence
Deployment	Flask REST API	Lightweight, easy integration with existing healthcare platforms
Monitoring	Evidently AI	Automated drift reports, open-source, easy GitHub Actions integration
Class balancing	Class weights	Minor class imbalance across 7 obesity levels; class weights ensure fair recall across all classes

Section 5: Individual Contribution Statements

Sahil — 105295665 — Project Manager / Scrum Master

I was responsible for coordinating the overall system blueprint and architecture direction, establishing the Trello board with sprint planning structure, connecting the data-to-model-to-output pipeline design, and reviewing all team members' technical contributions. I compiled all sections into the final report, wrote the project introduction and system overview, and managed the Canvas and GitHub submission process.

Seniru — 104786647 — ML Engineer

I was responsible for the ML model component design, including Random Forest and Decision Tree selection and justification, defining the model output format (predictions and logic), planning the retraining strategy and monitoring approach, writing the MLOps section of the report, and explaining the retraining and drift detection process. I will lead model training, evaluation, and API development in Sprints 1 and 2.

Parav — 105321434 — Data Engineer

I was responsible for identifying and sourcing the UCI Obesity dataset, performing exploratory data analysis on eating habits and physical condition features, designing the preprocessing and feature encoding pipeline, and contributing to the retraining strategy from a data quality perspective. I will lead UCI obesity dataset download, EDA, feature encoding, preprocessing, and class weight application in Sprint 1.

Nainika — 105321162 — MLOps / DevOps Lead

I was responsible for system integration design (how the user interacts with the deployed model), designing the UI and healthcare operator interface concept, planning the DevOps infrastructure (GitHub branching, CI/CD), writing the DevOps section of the report, and designing the deployment pipeline end-to-end. I will implement GitHub Actions, MLflow model registry, and Evidently AI monitoring in Sprints 2 and 3.

GenAI Declaration

This report used Perplexity AI as a generative AI tool for the following purposes:

- Drafting initial content and structure for all sections of this report
- Brainstorming sprint plan elements, user story formats, and acceptance criteria
- Improving the clarity, grammar, and professional tone of written content

All AI-generated content was reviewed, edited, and verified by team members to ensure accuracy, relevance, and alignment with our specific project context. Generative AI was not used for data collection, model training, code implementation, or any technical deliverables.

Proof of AI use: Screenshots of the Perplexity AI conversation are provided in Appendix A.

APA Reference:

Perplexity AI. (2026, April 15). Response to prompts regarding AI-powered obesity classification system design and project planning [AI-generated content]. Perplexity AI. https://www.perplexity.ai

Appendix A: GenAI Proof Screenshot

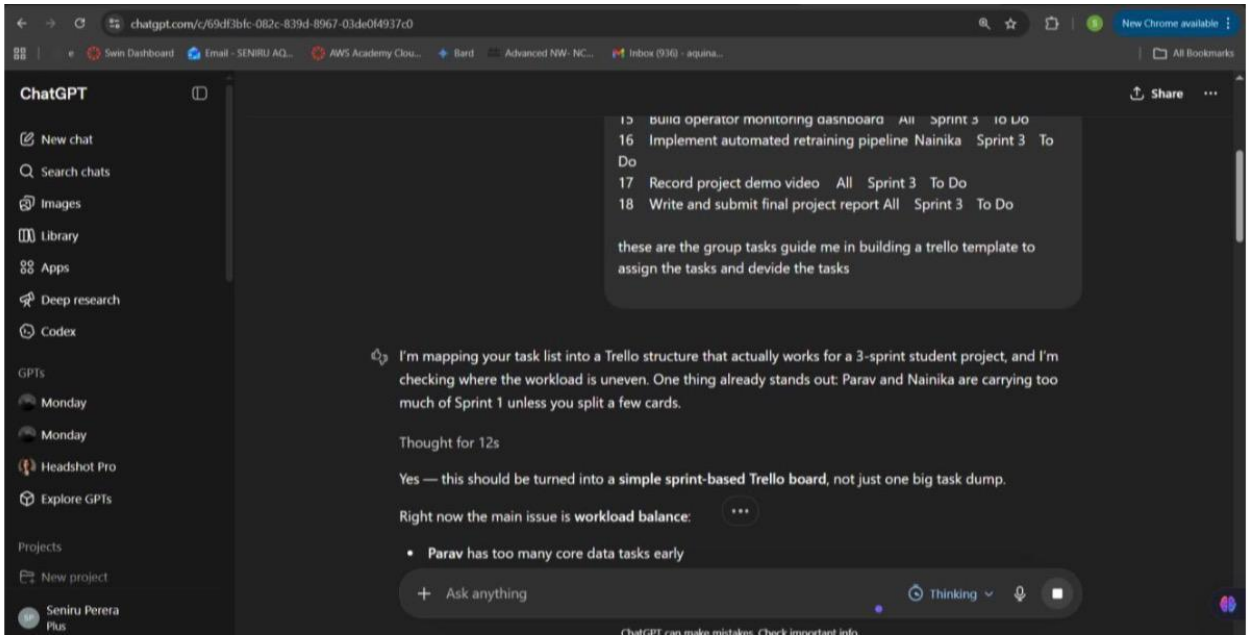


Image A1 — This screenshot shows Perplexity AI incorrectly assigning team member names to Trello cards. The student corrected this by providing actual Trello screenshots as evidence, demonstrating critical evaluation of AI output.

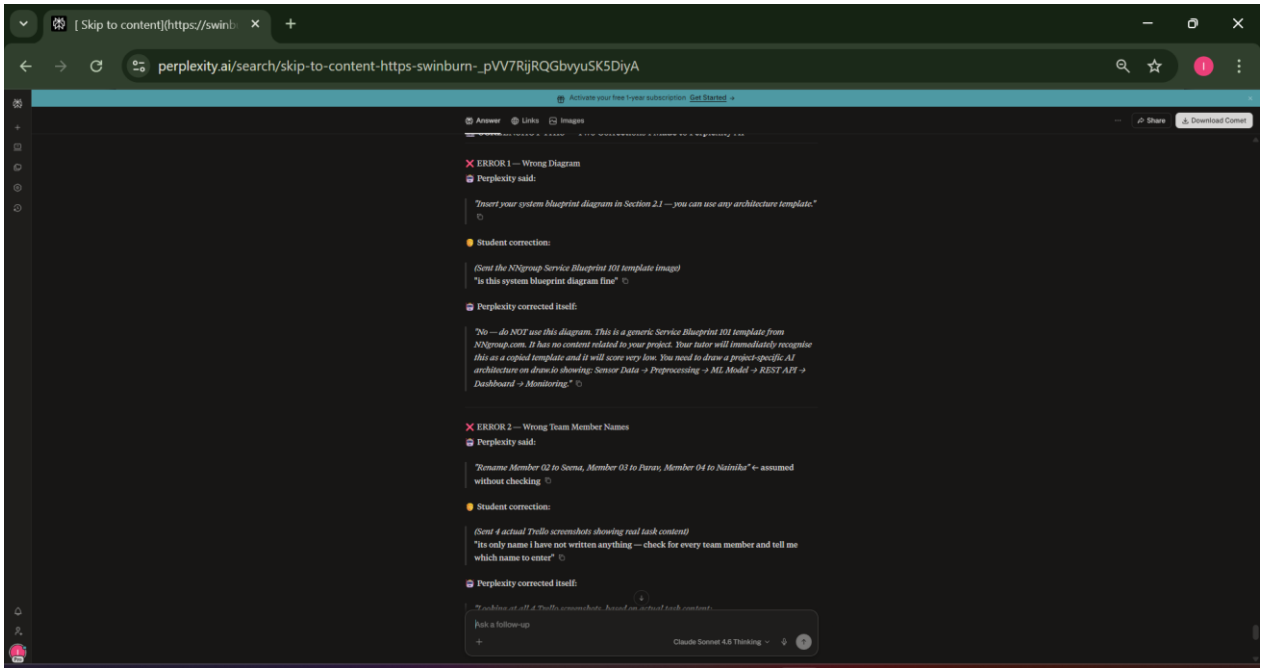


Image A2 — This screenshot shows the student directing Perplexity AI to correct the system architecture approach, demonstrating responsible and critical use of generative AI tools.